6.4.2 Authorizing requests using OAuth2

Suppose you store all your calendar data in Google Calendar. One day you discover a cool new startup app that uses your calendar data to determine when is the best time to schedule a massage. This is a revolutionary idea! No longer do you need to worry about scheduling your own massages—this new app will do it for you.

There is only one problem. This hot new startup app needs your Google Calendar data in order to work. How can the startup get data from a different company such as Google? Well, you could give your Google password to this startup app, and the startup app could then log into your Google account and read your calendar data.

Of course sharing your password is a horrible, horrible idea. Your password is the key not only to your calendar data, but to any data you store in Google. By giving your password to a third-party startup, you trust that third party not to do anything bad with your data (like accidentally delete your calendar entries). And even if you do trust them, their systems could be hacked and your password could land in the hands of a hacker. Passwords are just too valuable to share.

So if you can't share your password, how can the startup app gain access to your data? OAuth2 can save the day. OAuth2 is an open authorization standard. It's a popular authorization mechanism for APIs. APIs from Dropbox, Facebook, GitHub, and Google (to name a few) all use OAuth2. The goal of OAuth2 is to allow a site to get user data from a different site without asking for the user's password. This keeps the user's password safe, and puts the user in control of when and how his or her data is used.

If the startup app implements OAuth2, instead of asking for your password, it will redirect you to a special page hosted on Google. Figure 6.12 shows an example of what this authorization page looks like.

Google then asks you if it's okay to share your data with this hot new startup. If you click Accept, Google gives a token to the hot new startup app. The startup app can then include this token in the request to load Google Calendar data. No passwords need to be shared to make this work.

This token method is a much better option than sharing your password. The token is "scoped," meaning that it can only be used to load calendar data, and can't change or delete calendar data. And if you, the user, decide you don't want the startup app to access your data after all, you have the power to revoke the token. Revoking the token prevents the startup app from using the token to access your data.

Once the third-party app has a token, it can use the token to make requests to the Google Calendar API for a particular user's data. When making a request for data, the token is included in the Authorization header of the request, as shown in figure 6.13. Because the Authorization header is added to the request by the client (rather than passively included on requests like cookies), a malicious user can't trigger a cross-site request forgery attack.

If you're using OAuth2 with CORS, it's important to remember to whitelist the Authorization header on preflight requests. Because the Authorization header isn't a